

# Salesforce Aged Complex Org Revitalization Process

|                                                                    |          |
|--------------------------------------------------------------------|----------|
| <b>Scope of Work</b>                                               | <b>2</b> |
| <b>Strategic Triage, Tactical Remediation, and Cultural Change</b> | <b>3</b> |
| <b>Understanding the Current Mess</b>                              | <b>3</b> |
| <b>Clean It</b>                                                    | <b>4</b> |
| <b>Scale It</b>                                                    | <b>4</b> |
| <b>Future-Proof It</b>                                             | <b>4</b> |
| <b>Measurable Sprint Goals</b>                                     | <b>5</b> |
| <b>Tracking Progress</b>                                           | <b>5</b> |
| <b>High Level Goals</b>                                            | <b>5</b> |
| Security and Access Review                                         | 5        |
| Platform Health Assessment                                         | 6        |
| Data and Metadata Cleanup                                          | 6        |
| Performance Optimization                                           | 6        |
| Development Best Practices                                         | 6        |
| DevOps and Automation Enhancements                                 | 6        |
| Sandbox Strategy and Data Management                               | 7        |
| Monitoring and Governance                                          | 7        |
| 9. Strategic Evaluations                                           | 7        |
| User Experience Improvements                                       | 7        |
| <b>Detailed Steps to achieve above Goal</b>                        | <b>7</b> |
| Org-Wide Defaults (OWD) & Sharing Review                           | 7        |
| Optimizer & Health Check                                           | 8        |
| Static Code Analysis with PMD                                      | 8        |
| Identify Dead Code & Unused Flows                                  | 8        |
| Unused Fields & Objects                                            | 9        |
| Reports & Dashboards Cleanup                                       | 9        |
| Optimize SOQL Queries                                              | 9        |
| CPU Time & DML Optimization                                        | 10       |
| Large Data Volume (LDV) Strategies                                 | 10       |
| Data Archiving                                                     | 10       |
| AppExchange Packages Review                                        | 10       |
| Test Class Runner Implementation                                   | 11       |

|                                                         |    |
|---------------------------------------------------------|----|
| Identify Dead Code and Unused Flows                     | 11 |
| Cleanup Unused Fields and Objects                       | 11 |
| Cleanup Unused Reports and Dashboards                   | 12 |
| Improve SOQL and Query Optimization                     | 13 |
| CPU Time, SOQL, and DML Limit Reduction                 | 13 |
| Implement Large Data Volume (LDV) Management Techniques | 14 |
| Data Archival Strategy                                  | 14 |
| Review AppExchange Packages                             | 15 |
| Implement Test Class Runner and CI/CD Branding          | 15 |
| Sandbox Strategy, Seeding, and Data Masking             | 16 |
| Org Limit Monitoring and Optimization                   | 16 |
| Cleanup, Patterns, Documentation, and Best Practices    | 17 |
| Code and Flow Documentation                             | 17 |
| Design Patterns Enforcement                             | 17 |
| Standard vs Customization vs AppExchange                | 17 |

# Title: Salesforce Aged Complex Org Revitalization Plan

Author: Srini Pusuluri

## About Author

Srini Pusuluri - M.Tech IIT Kharagpur India

FMR Distinguished Scientist in Indian Space and Defence, Salesforce CRM and AI Architect

**Senior Salesforce (SFDC), AI, and CRM Program Architect**, highly skilled in integrating cutting-edge technologies like artificial intelligence and customer relationship management (CRM) platforms. With over **20 years of IT experience** (including 12 years in CRM/Salesforce and 5 years in AI), the author is a recognized leader in designing and delivering innovative AI and CRM solutions across industries. They have extensive expertise in **security, multi-org setups, data integration, design patterns, DevOps, and AI strategy**, and hold **20 Salesforce and 5 AI certifications**.

Feel free to reachout me for any AI/CRM free suggestions

<https://www.linkedin.com/in/pusulurisrinivasa/>



# Scope of Work

When managing a complex, high-volume Salesforce environment plagued by technical debt and performance issues, a structured, phased improvement plan is essential. The goal is to transform the org into a **scalable, efficient, and future-ready** platform



## Strategic Triage, Tactical Remediation, and Cultural Change

This document outlines a structured approach to addressing organizational challenges through strategic triage, tactical remediation, and fostering cultural change. By understanding the current state of affairs, implementing necessary clean-up actions, scaling solutions, and ensuring future-proofing, organizations can achieve sustainable improvements. The importance of communication and documentation is emphasized alongside technical fixes, and measurable sprint goals are established to track progress effectively.

# Understanding the Current Mess

## 1. Assessment Phase

- Conduct a comprehensive audit of existing processes and systems.
- Identify key pain points and areas requiring immediate attention.
- Gather feedback from stakeholders to understand their perspectives.

## 2. Documentation

- Create a detailed report of findings.
- Document current workflows, challenges, and inefficiencies.
- Establish a baseline for measuring improvements.

# Clean It

## 1. Immediate Remediation

- Prioritize issues based on impact and urgency.
- Develop a tactical plan to address high-priority problems.
- Implement quick wins to build momentum.

## 2. Communication

- Regularly update stakeholders on progress and changes.
- Ensure transparency in the remediation process.
- Foster an open dialogue for feedback and suggestions.

# Scale It

## 1. Systematic Improvements

- Identify successful remediation strategies and standardize them.
- Develop training programs to ensure team members are equipped to maintain improvements.
- Create a knowledge base for ongoing reference.

## 2. Documentation

- Update documentation to reflect new processes and systems.
- Ensure all changes are recorded for future reference and training.

# Future-Proof It

## 1. Sustainability Planning

- Establish metrics for ongoing performance evaluation.
- Implement regular review cycles to assess the effectiveness of changes.
- Adapt strategies based on evolving organizational needs and external factors.

## 2. Cultural Change

- Promote a culture of continuous improvement and accountability.
- Encourage collaboration and innovation among team members.
- Recognize and reward contributions to cultural transformation.

# Measurable Sprint Goals

## 1. Sprint 1: Assessment and Documentation

- Complete the audit of current processes within two weeks.
- Document findings and share with stakeholders by the end of the sprint.

## 2. Sprint 2: Immediate Remediation

- Identify and address at least three high-priority issues within two weeks.
- Communicate progress and gather feedback weekly.

## 3. Sprint 3: Systematic Improvements

- Standardize at least two successful remediation strategies within three weeks.
- Develop and launch training programs for team members.

## 4. Sprint 4: Sustainability and Cultural Change

- Establish performance metrics and review cycles within four weeks.
- Launch initiatives to promote a culture of continuous improvement.

# Tracking Progress

## ● Dashboards

- Create visual dashboards to track key performance indicators (KPIs) related to remediation efforts.
- Update dashboards weekly to reflect progress and areas needing attention.

## ● Weekly Reports

- Generate weekly reports summarizing progress, challenges, and next steps.
- Share reports with all stakeholders to maintain transparency and engagement.

By following this structured approach, organizations can effectively navigate their challenges, implement necessary changes, and foster a culture of continuous improvement, ensuring long-term success and resilience.

# High Level Goals



## **Security and Access Review**

- Evaluate Organization-Wide Defaults (OWD) and Sharing Rules.
- Ensure data security, minimize overexposure, and optimize access models.

## **Platform Health Assessment**

- Run Salesforce Optimizer Reports.
- Use static code analysis tools like PMD to identify code quality issues.
- Identify dead code, unused Apex classes, triggers, and abandoned flows.

## **Data and Metadata Cleanup**

- Clean up unused fields, custom objects, record types, page layouts, and reports.
- Archive and purge historical large object data using LDV (Large Data Volume) strategies.

## **Performance Optimization**

- Analyze SOQL queries using tools like Query Plan Analyzer.
- Redesign inefficient queries and optimize CPU, SOQL, and DML operations.
- Apply Governor Limit best practices.

## **Development Best Practices**

- Standardize code and flow documentation.
- Apply consistent naming conventions for Apex, Flows, and Integrations.
- Introduce and enforce scalable design patterns.

## **DevOps and Automation Enhancements**



- Implement a CI/CD pipeline tailored for Salesforce development.
- Set up automated test class runners and code coverage reports.
- Create a branding and release strategy for feature deployment.

## **Sandbox Strategy and Data Management**

- Plan effective sandbox environments for dev, QA, UAT, and training.
- Seed test data and apply data masking techniques to protect sensitive data.

## **Monitoring and Governance**

- Set up dashboards to monitor Salesforce limits and system health.
- Establish ongoing governance for platform changes.

## **Strategic Evaluations**

- Review existing AppExchange packages for optimization or retirement.
- Strategize when to use out-of-the-box features vs custom development vs third-party apps.

## **User Experience Improvements**

- Optimize Lightning Page performance.
- Prioritize using standard Salesforce functionality where possible to enhance reliability and reduce maintenance.

# Detailed Steps to achieve above Goal

## Org-Wide Defaults (OWD) & Sharing Review

- **Audit OWD settings:** Ensure they follow the principle of least privilege.
- **Evaluate sharing rules:** Identify and remove redundant or conflicting rules.
- **Assess role hierarchy:** Simplify overly complex hierarchies.
- **Review manual sharing:** Identify and document instances of manual sharing.
- **Document findings:** Create a report detailing current sharing settings and recommendations. [Reddit](#)

## Optimizer & Health Check

- **Run Salesforce Optimizer:** Identify unused fields, reports, and other metadata.
- **Conduct Health Check:** Assess security settings and compliance.
- **Analyze results:** Prioritize issues based on impact and effort.
- **Plan remediation:** Develop a plan to address identified issues.
- **Communicate findings:** Share results with stakeholders for transparency. [Validity](#)

## Static Code Analysis with PMD

- **Set up PMD:** Configure rulesets tailored to your org's coding standards.
- **Run analysis:** Identify code smells, unused variables, and other issues.
- **Prioritize fixes:** Focus on high-impact areas first.
- **Document technical debt:** Create a backlog of issues for remediation.

- **Integrate into CI/CD:** Ensure ongoing code quality checks.

## Identify Dead Code & Unused Flows

- **Inventory Apex classes/triggers:** Use tools like OrgCheck to find unused code.
- **Review Flows and Process Builders:** Identify and deactivate obsolete automations.
- **Clean up metadata:** Remove deprecated components.
- **Update documentation:** Reflect changes in system documentation.
- **Communicate changes:** Inform stakeholders of removals and reasons. [Reddit](#)

## Unused Fields & Objects

- **Run Field Trip:** Analyze field usage across objects.
- **Identify unused fields:** Mark fields with low or no usage.
- **Plan field deprecation:** Develop a strategy for safe removal.
- **Review custom objects:** Identify objects no longer in use.
- **Document changes:** Maintain a record of deprecated fields/objects. [Reddit](#)

## Reports & Dashboards Cleanup

- **Audit reports/dashboards:** Identify those not accessed in the last 6-12 months.
- **Consolidate duplicates:** Merge similar reports/dashboards.
- **Archive obsolete reports:** Retain for historical reference if needed.
- **Implement naming conventions:** Standardize for easier identification.
- **Train users:** Educate on best practices for report/dashboard creation. [MediumPort and Starboard+1XB Software+1](#)

## Optimize SOQL Queries

- **Review Apex code:** Identify inefficient queries.
- **Use Query Plan Tool:** Analyze query performance.
- **Refactor queries:** Optimize for selectivity and indexing.
- **Implement governor limit checks:** Prevent future performance issues.
- **Document query standards:** Establish best practices for developers.

## CPU Time & DML Optimization

- **Analyze Apex executions:** Identify methods exceeding CPU limits.
- **Refactor code:** Reduce nested loops and complex logic.
- **Bulkify operations:** Ensure code handles large data volumes efficiently.
- **Limit DML operations:** Combine operations where possible.
- **Monitor performance:** Set up alerts for CPU/DML limit breaches. [Me](#)

## Large Data Volume (LDV) Strategies

- **Identify LDV objects:** Focus on objects with millions of records.
- **Implement indexing:** Use skinny tables and custom indexes.
- **Optimize queries:** Use selective filters and avoid full table scans.
- **Archive old data:** Move historical data to external storage if needed.
- **Monitor performance:** Regularly assess query times and adjust strategies. [Metazoa](#)

## Data Archiving

- **Define archiving criteria:** Determine which data qualifies for archiving.
- **Develop archiving process:** Automate data movement to archives.

- **Ensure data integrity:** Validate that archived data remains accessible if required.
- **Comply with regulations:** Ensure archiving meets compliance standards.
- **Document archiving policy:** Provide guidelines for future data management.

## AppExchange Packages Review

- **Inventory installed packages:** List all third-party apps.
- **Assess usage:** Determine which packages are actively used.
- **Evaluate performance impact:** Identify packages causing issues.
- **Plan deprecation:** Schedule removal of unused or problematic packages.
- **Explore alternatives:** Consider native solutions or better-suited apps.

## Test Class Runner Implementation

- **Set up test frameworks:** Implement tools like Apex Test Execution.
- **Automate test runs:** Schedule regular test executions.
- **Monitor code coverage:** Ensure compliance with coverage requirements

## Identify Dead Code and Unused Flows

- **Run code coverage reports:** Identify Apex classes and triggers with no test coverage.
- **Analyze Flows:** Find inactive, deprecated, and duplicated flows.
- **List redundant classes/triggers:** Tag them for deprecation or deletion.
- **Document dependencies:** Check if "dead" code or flows are referenced indirectly.
- **Prepare decommissioning plan:** Schedule safe removals without breaking functionality.
- **Communicate impacts:** Inform business owners if any impacts are expected.

- **Set "to-be-archived" labels:** Tag old metadata for future cleanup.
- **Perform risk assessment:** Analyze if backup/versioning is needed.
- **Create a backup plan:** Backup metadata before deletion.
- **Plan phased removal:** Gradually phase out dead code.

## Cleanup Unused Fields and Objects

- **Run Field Trip / Field Utilization Reports:** Find rarely used fields.
- **Identify orphaned objects:** Find objects with minimal/no usage.
- **Validate with business:** Confirm if fields/objects are truly obsolete.
- **Mark for deletion:** Stage them in a sandbox first.
- **Check managed package fields:** Request ISV support if needed.
- **Backup metadata:** Export details for historical records.
- **Align with compliance/legal:** Especially for customer data fields.
- **Stage change sets:** Build packages for removing them.
- **Rollout cleanups in waves:** Avoid massive deletions at once.
- **Document changes:** Keep a cleanup log for audit.

## Cleanup Unused Reports and Dashboards

- **Identify unused reports:** Sort by last run date.
- **Find redundant dashboards:** Analyze overlaps in metrics.
- **Group by business unit:** Collaborate with teams to confirm deletions.
- **Archive first:** Move unused assets to "Archive" folders.
- **Set deletion policy:** After 90 days archived, then delete.

- **Rebuild critical dashboards:** Consolidate where necessary.
- **Implement naming conventions:** For easy future audits.
- **Update folder permissions:** Avoid open-access issues.
- **Document useful KPIs:** Consolidate in a KPI library.
- **Encourage self-service analytics:** Reduce custom report sprawl.

## Improve SOQL and Query Optimization

- **Enable Query Plan Tool:** Analyze slow SOQL queries.
- **Avoid Select \* (wildcards):** Query only needed fields.
- **Use indexed fields in WHERE clauses:** Optimize searches.
- **Reduce nested queries:** Flatten queries where possible.
- **Use LIMIT and OFFSET smartly:** In paginated data fetching.
- **Restructure queries:** Use child-to-parent queries smartly.
- **Apply skinny tables if needed:** For huge objects.
- **Use selective filters:** Especially on large data volumes (LDV).
- **Enable Query Caching:** In Lightning Experience.
- **Benchmark improvements:** Measure before vs. after changes.

## CPU Time, SOQL, and DML Limit Reduction

- **Refactor trigger logic:** Use Trigger Handlers/Frameworks.
- **Batch DML operations:** Aggregate inserts/updates.
- **Minimize synchronous calls:** Move to Queueable Apex or Batch.
- **Implement bulkification everywhere:** Avoid FOR loops inside DMLs.

- **Move non-critical operations async:** Post-commit processing.
- **Use Platform Events:** For scalable event-driven designs.
- **Analyze CPU spikes:** With Debug Logs/Monitoring.
- **Fix recursive triggers:** With static variables.
- **Reduce chained workflows:** Consolidate into single flows.
- **Educate developers:** Enforce best practices via review gates.

## Implement Large Data Volume (LDV) Management Techniques

- **Use skinny tables:** For wide objects.
- **Archive old records:** Move to external systems if not needed.
- **Use Big Objects:** For 50M+ record sets.
- **Implement partitioning strategies:** For huge datasets.
- **Enable Async SOQL:** For heavy queries.
- **Use custom indexes:** Request Salesforce support if needed.
- **Optimize sharing recalculations:** Defer where appropriate.
- **Design around governor limits:** Maximize query and DML efficiency.
- **Split transactions:** Reduce risk of limits exceeded.
- **Document all strategies:** For governance.

## Data Archival Strategy

- **Define archival criteria:** Based on object, date, or status.
- **Implement field-based filters:** "IsArchived" checkboxes, etc.
- **Create archive storage:** External system or Big Objects.



- **Develop archival jobs:** Batch Apex / Scheduled Jobs.
- **Maintain searchable archive:** If needed via APIs.
- **Communicate archival policy:** To stakeholders.
- **Ensure GDPR compliance:** For customer data.
- **Plan incremental archiving:** Avoid massive batch jobs.
- **Test archive recovery:** Validate that data can be restored.
- **Track storage savings:** Show quick wins to leadership.

## **Review AppExchange Packages**

- **Audit installed packages:** Usage, licensing, versions.
- **Remove redundant tools:** Save costs and complexity.
- **Ensure package upgrades:** Patch security holes.
- **Document dependencies:** Critical packages for solutions.
- **Identify unsupported apps:** Plan replacement.
- **Evaluate 3rd party alternatives:** Cheaper/better tools.
- **Assess managed vs unmanaged:** Manage security risks.
- **Run security assessments:** For external apps.
- **Define future review cadence:** Bi-annual evaluations.
- **Tag apps by purpose:** Monitoring, CPQ, integrations, etc.

## **Implement Test Class Runner and CI/CD Branding**

- **Deploy unified Test Class Runner:** Batch test runs.
- **Target 90%+ test coverage:** Clean up bad tests.

- **Set up DevOps CI/CD pipelines:** GitHub, Bitbucket, Copado, Gearset, or others.
- **Automate unit and regression testing:** PR-level enforcement.
- **Promote source-driven development:** Avoid Org-Driven risks.
- **Build modular pipelines:** Scratch org creation, validation, deployments.
- **Brand internal tools:** Dashboards for dev visibility.
- **Document deployment standards:** Mandatory DevOps best practices.
- **Enforce naming standards:** For pipelines/jobs.
- **Track deployment KPIs:** Success rates, mean time to deploy.

## **Sandbox Strategy, Seeding, and Data Masking**

- **Define sandbox types:** Dev, QA, UAT, Training.
- **Implement seeding tools:** Data Loader, OwnBackup, etc.
- **Automate data loads:** Sandbox refresh events.
- **Apply data masking:** PII protection in non-prod.
- **Version control metadata:** Keep sandboxes aligned.
- **Limit sandbox access:** Only needed profiles.
- **Sandbox naming convention:** Clear purpose labeling.
- **Sandbox refresh calendar:** Regular cycle definition.
- **Monitor sandbox storage:** Avoid overages.
- **Govern sandbox cloning:** Approval process.

## **Org Limit Monitoring and Optimization**

- **Set up limit monitoring tools:** Salesforce Limits API.

- **Track critical thresholds:** API calls, storage, concurrent jobs.
- **Implement proactive alerts:** Before hitting limits.
- **Optimize API usage:** Bulk API 2.0 for loads.
- **Reduce record lock issues:** Rethink transaction design.
- **Review platform events limits:** Check critical integrations.
- **Plan async migration:** For long-running operations.
- **Conduct limit stress testing:** Identify bottlenecks.
- **Enable event monitoring:** If available.
- **Create org health dashboards:** Weekly review.

## Code and Flow Documentation

- **Document Apex classes/triggers:** Architecture diagrams + code purpose.
- **Document Flows:** Visual maps + descriptions.
- **Maintain ERD diagrams:** Updated data model maps.
- **Publish developer guides:** Naming standards, Do's/Don'ts.
- **Set flow limits:** Number of elements, nesting levels.

## Design Patterns Enforcement

- **Apex patterns:** Factory Pattern, Service Layer, Selector Layer.
- **Flow patterns:** Modular Subflows, Fault handling.
- **Integration patterns:** Platform Events, Change Data Capture, Streaming API.
- **Trigger Frameworks:** 1 trigger per object.

- **Queueable/Batched/Aync Apex:** Best practices enforced.

## **Standard vs Customization vs AppExchange**

- **Use Out-of-the-Box first:** Lightning Components, Flows, Reports.
- **Customize carefully:** If OOTB fails or is inefficient.
- **Evaluate AppExchange:** For capabilities not feasible internally.
- **Minimize technical debt:** Always prefer standard where possible.